Subiect 1

1. Răspuns corect (**D**) - Altceva. Codul nu va compila pentru ca metoda *Create* din clasa *Student* este **statica** şi nu poate accesa membri a unei instante (în cazul de fata membri *Name* respectiv *Grade*). NU se acorda punctaje parțiale.

2. Răspuns corect (**E**) - Altceva. Codul nu va compila pentru ca obiectul *t* de tipul *Test* este creat folosind constructorul cu un parametru de tipul char (*(x+1) se va traduce într-un obiect de tipul char). Însă constructorul cu parametru de tipul char a fost explicit şters din definiţia clasei Test (`Test(char) = delete;`) prin urmare codul nu va compila. NU se acorda punctaje parţiale.

3. Răspuns corect (**D**) - Altceva. Metoda *Set* din clasa *Test* este *const* (deci nu poate modifica nici un membru al instantei). Însă poate modifica orice valoare a parametrilor pe care ii primeşte.Cum metoda *Set* a fost apelata cu un parametru care este chiar obiectul, în realitate poate (prin acel pointer) sa ii modifice membru *value.* Acest lucru înseamnă ca apelul (`Test t; t.Set(&t, 3);)` va seta t.*Value* la valoarea 3. Apelul *t.Get()* va returna 3 << 3 (adică 00000011 shiftat 3 poziţii → 00011000 = **24**). Se acorda 2 puncte dacă se ajunge la concluzia ca se afişează 3 << 3 dar nu se face calculul corect (24).

4. Variabila *counter* este globala (deci e initializata cu 0). La fiecare apel al unui constructor de tipul *Number* ea este incrementata. Acest lucru înseamna ca atunci când vom crea un vector de tipul *Number* (`Number *n = new Number[10];)` apelul constructorului se va face de 10 ori iar n[0].*v* = 5, n[1].*v* = 4, s.a.m.d. Destructorul clasei *Number* are operatorul *noexcept* (false) deci poate arunca excepţii. Apelul (`delete []n;)` va apela destructorul pentru fiecare element din n pana când se arunca o excepţie, moment în care se va opri. Apelul destructorului se realizeaza in ordine inversa (de la ultimul element la primul). In acest caz codul compilează si va afişa: -4, -3, -2, -1**.** Se punctează corect şi dacă se da răspunsul 5,4,3,2,1 (împreună cu explicaţia de mai sus). NU se acorda punctaje parţiale şi nu se punctează dacă afişarea secventei de afişare nu este însoţită de explicaţia de mai sus.

5. Răspuns corect (**E**) - Altceva. *f3(1)* returnează un ***int*** dedus din tipul de return cu valoarea 1 << 4 = **16**. *f1(f3(1))* ⇔ *f1(16)* si va returna un char (dedus din definiţia funcţiei lambda (`auto f1 = [](int x)->char {return x * x; }`) adica 16 * 16 = 256. Însă 256 nu poate fi reprezentat pe 8 biţi (dimensiunea unui char) asa ca va fi trunchiat. 256 = 1**00000000** deci 16*16 pe 8 biti va da **0.** *f2(f3(f1(1)))* ⇔ *f2(0)*

care se evalueaza in 0 * 0 = 0. Răspuns corect - va afișa 0. Dacă se dă răspunsul C (65536) și este corect explicat calculul pana la 65536 (dar nu se tine cont de cast-ul la char) se acordă 2p).

6. Se pot face mai multe implementari pentru clasa MyList. Exemplu:

```
class MyList
{
   int v[5];
public:
   explicit MyList(int start, int step)
   {
      for (int tr = 0; tr < 5; tr++)
         v[tr] = start + tr * step;
   }
   int* begin() { return &v[0]; }
   int* end() { return &v[5]; }
};
```

Se puncteaza:

- 3pct - utilizarea lui ***begin()*** și ***end()*** astfel încât codul din ***main*** pentru snipet-ul 1 sa funcţioneze (doar crearea unor funcţii ***begin()*** și ***end()*** dar care sa nu funcţioneze în contextului codului din ***main*** nu se punctează)
- 1pct - constructorul sa fie precedat de operatorul explicit sau dacă se face un constructor cu std::initializer_list<int> care este șters (cu delete) sau privat. [ex: `MyList(std::initializer_list<int> i) = delete;`]

7. Se puncteaza:

a) 1 pct - definiţia corectă a *Principiului inversării dependenţelor*:

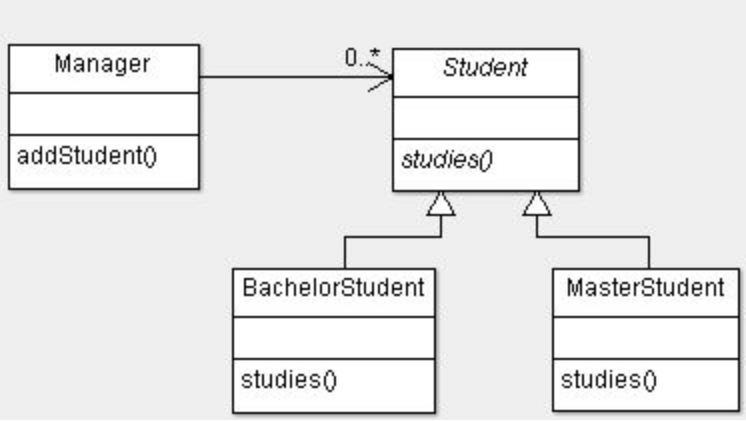
A. Modulele de nivel înalt nu trebuie sa depindă de modulele de nivel jos. Amândouă trebuie să depindă de abstracţii.

B. Abstracţiile nu trebuie sa depindă de detalii. Detaliile trebuie să depindă de abstracţii.

b) 1 pct - explicaţia corectă:

Clasa de nivel înalt Manager depinde de clasele de nivel jos BachelorStudent și MasterStudent. Ar trebui să existe câte o metodă de adăugare pentru fiecare clasă de nivel jos.

c) 1 pct - diagrama



d) 1 pct - codul:

```
class Manager {
  public: void addStudent(Student);
  private: list<Student> *students;
}
class Student {
  virtual void studies() = 0;
};

class BachelorStudent : public Student {
  virtual void studies();
};

class MasterStudent : public Student {
  virtual void studies();
};
```

Subiect 2

1. Răspuns corect (**C**) - Codul nu compilează. Codul nu va compila deoarece constructorul default din clasa ***A*** a fost şters (`A() = delete;`). Cum constructorul default din clasa ***B*** nu apelează explicit nici un alt constructor din clasa ***A***, se va încerca apelul constructorul default din clasa ***A*** care fost şters. Compilatorul va emite o eroare în acest caz. NU se acorda punctaje parţiale.

2. Răspuns corect (**E**) - Altceva. Codul nu va compila pentru ca un constructor NU poate fi static (`static Data() { std::cout << "D,"; }`). NU se acordă punctaje parţiale.

3. Răspuns corect (**D**) - Altceva. Clasa *A* are o funcţie virtuala (*Print*) deci are si vfptr. Deasemenea, clasa *A* suprascrie operatorul new. Ceea ce înseamnă ca orice obiect de tipul A se va afla la aceeaşi locaţie (*& Buffer [0]*). Cum clasa *B* este derivata din clasa *A* si nu suprascrie operatorul new, acelaşi lucru va fi valabil şi pentru obiectele de tipul B alocate prin utilizarea operatorului new (se vor găsi la aceeaşi locaţie toate : (*& Buffer [0]*). In urma execuţiei codului din *main* se vor întâmpla următoarele lucruri:
   - [`A* a1 = reinterpret_cast<A*>(new B());`] → a1 va avea vfptr-ul la adresa Buffer[0]. Cum obiectul e de tip B, vfptr-ul va fi de tipul B
   - [`A* a2 = new A();`] → a2 va avea vfptr-ul la aceeaşi adresa (Buffer[0]). Cum vfptr se setează la apelul constructorului, crearea lui a2 va suprascrie şi vfptr-ul lui a1 (pentru ca se găsesc la aceeaşi adresa). Deci după execuţia acestei instrucţiuni a1 şi a2 au acelaşi vfptr (cel a lui a2 care e de tipul A)
   - [`a1->Print(); (*a1).Print();`] → vfptr-ul este de tipul A, se va afisa **AA**
   - [`B* b1 = new B();`] → similar se crează un obiect de tipul B şi se suprascrie vfptr-ul de la &Buffer[0] cu unul de tipul B
   - [`b1->Print(); a2->Print();`] → vfptr-ul este de tipul B, se va afisa **BB**

   Răspuns corect (**D**) - Afişează AABB. NU se acorda punctaje parţiale si nu se puncteaza raspunsul AABB fara explicatia de mai sus.

4. Răspuns corect (**D**) - Altceva. Funcţia (`Suma<int,Number,double>(Number(10))`) va returna (conform template-ului) (T1)(x+y). In cazul de fata T1 = int, x este de tipul Number, iar y ia valoarea default din template adică T1(2.5) = int(2.5) = 2, valoare convertita la T3 (adică la double = 2.0). Deci rezultatul va fi (int)(Number(10) + 2.0). Clasa Number supraîncarcă operatorul de adunare cu double, deci Number(10)+2.0 = (float)(10 * 2.0+1) / 2.0f = 21.0 / 2.0f = 10.5f.

Template-ul casteaza la T1 care este int, deci funcţia Suma va returna int(10.5) = 10. Răspuns corect (**D**) - Afişează 10. NU se acorda punctaje parţiale şi nu se punctează răspunsul 10 fără explicaţia de mai sus.

5. **mutable** din definiţia Lambda înseamnă ca ceea ce se captează poate fi utilizat în continuare a o variabila interna în clasa reprezentata de expresia Lambda. Variabila interna "a" din lamba nu are nici o legătura cu copia sa variabila "a" definita local. Deci, se vor executa următoarele acțiuni:
   - [`auto f = [a](int x, int y) mutable { return (a += x + y) * 2; };`] → a local ramane 0, a din Lambda este si el 0.
   - [`a += f(1, 2);`] → a din Lambda se modifica si devine 0 + 1 + 2 = 3, functia lambda returneaza 3 * 2 = 6, a-ul local devine 0+6 = 6
   - [`std::cout << f(3, 4) << "," << a;`] → f(3,4) va returna (3 (a-ul din lambda) + 3 + 4) * 2 = 10* 2 = 20, iar a-ul local este 6.

Răspuns corect Afişează **20,6**. Se acord 2 puncte pentru a=6, respectiv pentru valoarea lui a din Lambda = 20.

6. Răspuns corect:

```
void PrintValue(int n)
{
       switch (n)
       {
       case 2: throw int(2); break;
       case 4: throw char(48 + 4); break;
       case 6: throw "Number six"; break;
       default: std::cout << n << ",";
       }
}
```

Se acorda (1 pct pentru cazul cu 2, 1 pct pentru cazul cu 4, 1 pct pentru cazul cu 6 si 1 pct pentru valoarea default). Se accepta si forme echivalente cu un switch (ex: if).

7. Se punctează:
   a) 1 pct - definiţia corectă pentru *Principiul Substituirii al lui Liskov*
   Stabileşte când tipul unei clase este sutip a celui definit de altă clasă: dacă tipul clasei A este un subtip al celui definit de clasa B, atunci o referinţă de tipul A poate apare în orice context orice context în care o referinţă de tip B este aşteptată, fără a modifica corectitudinea program. Cazul de interes este atunci când A este derivată din B.
   b) 1 - pct explicaţia corect:
   O instanţă a clasei OversizedLorry nu poate fi utilizat oriunde este aşteptată o instanţă a clasei Lorry. Exemplu:

   bool Lorry::isLegalSpeed()

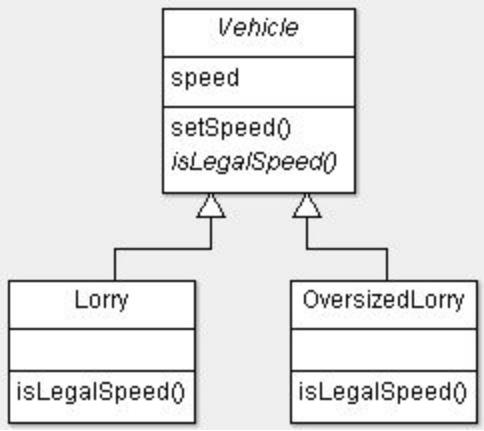
```
  return speed <= 90;
}

bool OversizedLorry::isLegalSpeed()
  return speed <= 70;
}

Lorry lorry;
lorry.setSpeed(90);
b = lorry.isLegalSpeed(); // b == true

OverSizedLorry overslorry;
overslorry.setSpeed(90);
b = overslorry.isLegalSpedd; // b == false
```

c) 1 pct - diagrama



d)

```
class Vehicle {
  public:
    void setSpeed();
    virtual bool isLegalSpeed() = 0;
  private:
    int speed;
};
class Lorry : public Vehicle {
  public:
     virtual bool isLegalSpeed();
};
class OversizedLorry : public Vehicle {
  public:
     virtual bool isLegalSpeed();
};
```